

AstraNotes Test Improvement Log

1. Feature / Requirement Reviewed

Feature: Slice C — search notes by title or body text. Requirements: FR-8 (search ranked by recency), NFR-1 (≤ 100 ms p95 at 10k notes — performance target, not asserted here), SEC-1 (SecureNote bodies excluded from search until unlocked).

2. Original (Weak) Test

The test as it stood after PR-A merged:

```
from unittest.mock import MagicMock
from astranotes.repository.sqlite_repo import LocalSQLiteRepository
from astranotes.services.note_service import NoteService
from astranotes.services.validation import ValidationService

def test_search_returns_matches_ranked_by_recency(tmp_path):
    repo = MagicMock(spec=LocalSQLiteRepository)
    repo.search.return_value = [
        ("id-3", "alpha note three", "2026-05-20T10:00:00+00:00", "TEXT"),
        ("id-2", "alpha note two", "2026-05-20T09:00:00+00:00", "TEXT"),
        ("id-1", "alpha note one", "2026-05-20T08:00:00+00:00", "TEXT"),
    ]
    svc = NoteService(repo=repo, validation=ValidationService(), audit=MagicMock())

    results = svc.search("alpha")

    assert results == [
        ("id-3", "alpha note three", "2026-05-20T10:00:00+00:00", "TEXT"),
        ("id-2", "alpha note two", "2026-05-20T09:00:00+00:00", "TEXT"),
        ("id-1", "alpha note one", "2026-05-20T08:00:00+00:00", "TEXT"),
    ]
```

The test is green. It is also nearly useless. Four reasons:

3. What Was Wrong — Test Smell Breakdown

Smell	Where it shows up	Why it's a problem
-------	-------------------	--------------------

Brittleness	<code>assert results == [...]</code> asserts full ordered equality	FR-8 says "ranked by recency" but FTS5 BM25 + identical secondary keys can return ties in nondeterministic order. The test will fail intermittently as soon as two notes share an <code>updated_at</code> second, and it will fail for the wrong reason — the user-observable behavior (recent notes appear) is still correct.
Over-mocking (deck's "Fake confidence" red zone)	<code>repo =</code> <code>MagicMock(spec=LocalSQLiteRepository)</code> and <code>repo.search.return_value = [...]</code>	The test mocks the layer it is supposed to be exercising. It proves only that <code>NoteService.search()</code> returns whatever the repo returns — but the whole point of testing FR-8 is to catch a regression in the FTS5 query itself (the column list, the MATCH clause, the ORDER BY). A bug there would not fail this test.
Coupling to implementation shape	Asserts the four-tuple (id, title, <code>updated_at</code> , kind) exactly	If <code>LocalSQLiteRepository.list()</code> ever adds a <code>body_preview</code> column or changes the column order, every search test breaks even though the user-visible behavior is unchanged. The test couples to the transport of the result rather than the observable contract.

Failure mode masked (coverage gap)	No SecureNote in the seed data	SEC-1's acceptance criterion is "body content of locked SecureNotes shall not contribute to search results until the user unlocks them". The test never exercises this. A regression that started indexing SecureNote plaintext would pass this test cleanly. That is the worst kind of green test — green when the most consequential rule is broken.
------------------------------------	--------------------------------	--

The deck's framing applies: coverage is a clue, not a proof. This test contributes one green tick to the coverage report and tells us almost nothing about whether the search feature actually works.

4. Improved Test

```
import pytest
from unittest.mock import MagicMock
from astranotes.models.note import Note, NoteKind
from astranotes.repository.sqlite_repo import LocalSQLiteRepository
from astranotes.services.note_service import NoteService
from astranotes.services.validation import ValidationService

@pytest.fixture
def seeded_repo(tmp_path):
    """Real SQLite + FTS5 with a known seed set."""
    repo = LocalSQLiteRepository(tmp_path / "notes.db")
    svc = NoteService(repo=repo, validation=ValidationService(), audit=MagicMock())
    svc.create(title="alpha note one", body="alpha and beta", kind=NoteKind.TEXT)
    svc.create(title="alpha note two", body="alpha only", kind=NoteKind.TEXT)
    svc.create(title="alpha note three", body="just alpha here", kind=NoteKind.TEXT)
    svc.create(title="zeta", body="no match", kind=NoteKind.TEXT)
    # A SecureNote whose plaintext WOULD match — must be excluded (SEC-1).
    svc.create(title="locked diary", body="alpha alpha alpha", kind=NoteKind.SECURE)
    return repo, svc

def test_search_returns_text_matches_by_membership(seeded_repo):
    _, svc = seeded_repo
```

```

results = svc.search("alpha")
titles = {row[1] for row in results}
assert {"alpha note one", "alpha note two", "alpha note three"} <= titles, (
    "FR-8: every text note whose body or title contains the query must be returned"
)
assert "zeta" not in titles, "non-matching notes must not leak in"

```

```

def test_search_excludes_secure_note_bodies(seeded_repo):
    _, svc = seeded_repo
    titles = {row[1] for row in svc.search("alpha")}
    assert "locked diary" not in titles, (
        "SEC-1: SecureNote plaintext must never contribute to search results"
    )

```

```

def test_search_orders_by_recency_when_explicit(seeded_repo):
    repo, svc = seeded_repo
    results = svc.search("alpha")
    updated = [row[2] for row in results]
    assert updated == sorted(updated, reverse=True), (
        "FR-8: results ranked by recency (updated_at DESC). The repository's SQL "
        "must use explicit ORDER BY rather than relying on insertion order."
    )

```

Three changes the deck would call high signal:

1. Real persistence. LocalSQLiteRepository is constructed against tmp_path, so the FTS5 wiring, the schema, and the ORDER BY are all genuinely exercised. The bug the original test could not catch (FTS5 column omission) would now fail loudly.
2. Membership over ordering. The first test asserts a subset relation, not exact equality — it survives the BM25 tie-break nondeterminism and still proves the requirement (every matching note appears).
3. SEC-1 has its own test. The SecureNote exclusion is the most consequential rule for search; it now has a named assertion with a comment that quotes the requirement.

One thing I deliberately did not do: I did not assert against absolute timestamps or row counts. The test should fail when behavior is wrong, not when seed timing shifts.

5. Mocking Decision — Helping or Hiding?

The original test mocked LocalSQLiteRepository. That mock hid the real risk: the FTS5 SQL is the thing most likely to regress, and mocking it means the test cannot see those regressions. Per the deck's mocking spectrum, that is the red zone. The improved test moves into the deck's green zone:

What	Mocked?	Why
LocalSQLiteRepository	No. Real SQLite against tmp_path.	This is the layer under test. Mocking it would defeat the purpose. The tmp_path fixture gives perfect isolation without losing the FTS5 path.
AuditLogService	Yes (via MagicMock()).	Audit-log behavior is orthogonal to search correctness — its own test covers that. Mocking it here keeps the test focused.
ValidationService	No. Real instance.	Validation is cheap and pure; using the real one increases realism without adding cost.

The principle: mock the things that are not under test and are slow, flaky, or stateful. Do not mock the thing the test is about. In this case, the thing under test is the search path, so the repository stays real.

6. Coverage Gap That Still Matters

Even the improved test set above does not cover the NFR-1 latency claim — search returns ranked results in under 100 ms p95 with a 10,000-note collection on commodity hardware. The Sprint Zero plan calls this out as S0-11, the performance spike. The improved test exercises correctness (FR-8 + SEC-1) on a 5-note dataset; it cannot tell us whether the same query takes 30 ms or 3000 ms at scale.

This gap is deliberate, not an oversight. The deck's framing: coverage that doesn't tell you what you need to know is misleading coverage. Folding a 10,000-row benchmark into the unit/integration suite would slow CI dramatically and would not catch a perf regression any earlier than a dedicated perf job would. The right home for that test is a separate tests/perf/test_search_latency.py that runs nightly or on a [perf] label, not on every PR.

Other gaps I considered and deliberately did not close in this iteration:

- Edit + search. What happens to the index when a note is edited? Closer to an FR-2 integration test than an FR-8 search test; belongs in the FR-2 set.
- Delete + search. Soft-delete via FR-3's recoverable trash means the note should disappear from search but remain reachable via trash. Belongs in the FR-3 set.
- Query injection. MATCH 'alpha'; DROP TABLE notes; — FTS5 parameterization handles this, but a defensive test would still be worth adding. Logged as a follow-up.